Learning Parametrization with Implicit Neural Representations

Google Summer of Code 2026 • ML4SCI / GENIE

175 hours • Intermediate/Advanced

Name	Soumya Vajahhala
Email	soumyarao2810@gmail.com
Phone	+1 332-254-9093
GitHub	github.com/V-SOUMYA
University	Rutgers University, New Brunswick, NJ
Degree	M.S. Computer Science (Expected 2026)
Undergrad	B.Tech CS with ML Specialization, JUET (CGPA 8.0/10)
Time Zone	EST (UTC−5)
Mentors	Sergei Gleyzer, Ali Hariri, Amal Saif
Duration	175 hours (Standard)

1. Why I am Interested in This Project

I am not a particle physicist. But this project is about something I find genuinely interesting: what if we stored data as a function instead of a grid?

That question started for me during my fruit-detection project. I built a CNN on 224x224 images and it worked well. But I kept thinking: the fruit does not exist as pixels. It is a shape in space. Pixels are just one way to sample it. What if we learned the shape directly?

That led me to NeRF, then SIREN, then Fourier Feature Networks. I built a small coordinate MLP just to test the idea. It worked.

This project is the same idea on data that actually matters. Jet events at the LHC are energy fields. Right now, they get stored as 125x125 grids. An INR stores the same field as a small neural network that you can query at any point. That is a cleaner, more flexible representation. I want to build it.

2. Background

2.1 The problem with fixed grids

Each LHC detector records the same jet event differently:

- ECAL and HCAL record energy on a fixed grid (like an image).
- Tracking detectors record particle paths as scattered 3D points.

The current approach converts everything to a 125x125 pixel image. It works, but there are three issues:

- **Fixed resolution.** You cannot query between pixels. Upsampling is always lossy.
- **Sparse data.** Most pixels are zero. The grid wastes space on emptiness.
- **One geometry.** A model trained on ECAL images does not transfer easily to tracking data.

An INR avoids all three. The representation is a function, not a grid.

2.2 What is an INR?

An INR is a neural network that maps coordinates to values:

$$f_\theta(\eta, \phi) \to E$$

The network takes a detector coordinate and outputs the energy there. The whole event lives in the weights θ.

The problem with a plain MLP is that it tends to smooth out sharp edges. A jet has a dense core with sharp energy spikes. The fix is a positional encoding that expands the input before the MLP sees it:

$$\gamma(x) = [\sin(2\pi Bx),\ \cos(2\pi Bx)], \quad B \sim \mathcal{N}(0, \sigma^2)$$

This is called a Fourier Feature encoding. It lets the network learn fine detail without blurring.

2.3 My relevant background

- **PyTorch training loops.** I built a MobileNetV2 classifier from scratch for my B.Tech project. I know how to train, debug, and evaluate models.
- **Coordinate MLPs.** I implemented a Fourier Feature Network on my own after reading the NeRF and Fourier Feature papers. That is the closest thing I have done to this project.

- **Writing production code.** At Accenture I built Python tools that other engineers reviewed and used. I learned to write code that is readable, not just correct.
- **Math.** Linear algebra and probability from both B.Tech and Rutgers. I understand why positional encodings work.

3. Technical Plan

3.1 The central question

This project is not just about building an INR. It is about comparing it to existing methods. The core experiment is:

Does an INR give a better representation than an autoencoder or a GNN?

I will answer this with two metrics:

1. **Reconstruction quality.** How faithfully does each method reproduce the original event?
2. **Classification accuracy.** How useful is each representation for quark/gluon classification?

I already have working implementations of the autoencoder (Common Task 1) and the GNN (Common Task 2). So I start the summer with two baselines ready.

3.2 Step 1: Preparing the data

Each image becomes a list of (coordinate, energy) pairs. The code below does this conversion.

Listing 1: Convert a jet image into coordinate-value pairs

```python
import numpy as np

def image_to_pairs(image):
    # image is shape (125, 125)

    # find pixels with energy > 0
    rows, cols = np.nonzero(image)
    coords = np.stack([rows, cols], axis=1)   # (N, 2)
    values = image[rows, cols]                # (N,)

    # also sample some empty pixels
    # so the model learns to output zero there
    n = len(rows)
    all_idx = np.argwhere(image == 0)
    chosen = all_idx[np.random.choice(len(all_idx), n)]
    zero_vals = np.zeros(n)

    coords = np.vstack([coords, chosen])
    values = np.hstack([values, zero_vals])

    # scale coordinates to [-1, 1]
    coords = coords / 62.5 - 1.0

    return coords, values
```

> Why include zero pixels? If we only train on non-zero pixels, the model never learns that empty regions should output zero. Including a sample of zeros fixes this.

3.3 Step 2: The INR model

Listing 2: Fourier Feature INR in PyTorch

```python
import torch
import torch.nn as nn
import numpy as np

class INRModel(nn.Module):
    def __init__(self):
        super().__init__()

        # Random projection matrix (not trained, stays fixed)
        # Shape: (256, 2) -- maps 2D coords to 256 dimensions
        B = torch.randn(256, 2) * 10.0
        self.register_buffer('B', B)

        # MLP: input is 512 (sin + cos of 256 features)
        self.net = nn.Sequential(
            nn.Linear(512, 256), nn.ReLU(),
            nn.Linear(256, 256), nn.ReLU(),
            nn.Linear(256, 256), nn.ReLU(),
            nn.Linear(256, 1),   nn.Sigmoid()
            # Sigmoid keeps output in [0, 1] like our energy values
        )

    def encode(self, x):
        # x: (batch, 2) coordinates
        proj = 2 * np.pi * (x @ self.B.T)
        return torch.cat([torch.sin(proj), torch.cos(proj)], dim=-1)

    def forward(self, x):
        return self.net(self.encode(x)).squeeze(-1)
```

3.4 Step 3a: Fitting one INR per event

We train a separate network for each event. The trained weights become the event's representation.

Listing 3: Training loop for one event

```python
def fit_one_event(coords, values):
    model = INRModel()
    opt = torch.optim.Adam(model.parameters(), lr=5e-4)

    coords_t = torch.tensor(coords, dtype=torch.float32)
    values_t = torch.tensor(values, dtype=torch.float32)

    for step in range(2000):
        # pick 512 random points from this event
        idx = torch.randperm(len(coords_t))[:512]
        pred = model(coords_t[idx])
        loss = ((pred - values_t[idx]) ** 2).mean()

        opt.zero_grad()
        loss.backward()
        opt.step()

    # flatten all weights into one vector
    weights = torch.cat([p.data.flatten() for p in model.parameters()
        ])
    return weights  # this IS the event representation
```

3.5 Step 3b: Hyper-network (faster alternative)

Per-event fitting is slow for large datasets. A hyper-network predicts the INR weights from the image in one pass.

Listing 4: Hyper-network: image goes in, INR weights come out

```
class HyperNet(nn.Module):
    def __init__(self, n_weights):
        super().__init__()

        # encode the 3-channel jet image into a small vector
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),                          # -> 62x62
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),                          # -> 31x31
            nn.AdaptiveAvgPool2d(4),                  # -> 4x4
            nn.Flatten(),
            nn.Linear(64 * 16, 256)                   # -> latent z
        )

        # predict INR weights from the latent vector
        self.head = nn.Linear(256, n_weights)

    def forward(self, image):
        z = self.encoder(image)       # (batch, 256)
        weights = self.head(z)        # (batch, n_weights)
        return weights, z
```

The latent vector **z** can be used directly as a feature for classification.

3.6 Step 4: Evaluation

Reconstruction metrics (how faithful is the INR?):

- MSE: mean squared error per channel.
- PSNR: higher is better; intuitive in image terms.
- SSIM: captures structure, not just pixel error.
- Visual check: original vs. reconstructed, side by side.

Classification (does the representation carry physics?):

Listing 5: Using INR weights to classify quark vs gluon

```
# Step 1: collect representations for all events
all_features, all_labels = [], []
for image, label in dataset:
    coords, values = image_to_pairs(image)
    feat = fit_one_event(coords, values)  # or use hyper-net
    all_features.append(feat)
    all_labels.append(label)

# Step 2: train a simple classifier
clf = nn.Sequential(
    nn.Linear(feat.shape[0], 128),
    nn.ReLU(),
    nn.Linear(128, 2)
)
train(clf, all_features, all_labels)
```

```

# Step 3: compare AUC to baselines
# INR features -> auc_inr
# Autoencoder latent -> auc_ae    (already built)
# GNN features -> auc_gnn         (already built)
```

Ablations I will run:

- With vs. without Fourier encoding.
- Fourier scale $\sigma \in \{1, 5, 10, 20\}$.
- MLP depth: 2, 3, 4, or 6 layers.

4. Timeline

Priority note: Phases 1 and 2 are the core deliverable. Everything after that is an extension. If something runs over time, the buffer absorbs it.

Phase	Dates	Hours	Deliverable
Community Bonding	May 1–26	10 h	Meet mentors, finalize plan, set up repo.
Phase 1: Data + INR	May 27–Jun 16	30 h	Data pipeline, base INR, first reconstruction results.
Phase 2: Hyper-network	Jun 17–Jul 7	35 h	Hyper-network, fast inference, comparison to Phase 1.
Midterm Check	~Jul 14	5 h	Working INR, reconstruction metrics, midpoint report.
Phase 3: Classification	Jul 8–Jul 28	30 h	Classifier on INR features, AUC vs. AE and GNN baselines.
Phase 4: Ablations	Jul 29–Aug 18	30 h	Encoding ablation, depth sweeps, notebooks, documentation.
Final Submission	Aug 19–25	15 h	Final report, clean README, GSoC submission.
Buffer	Throughout	20 h	Unexpected issues, mentor feedback, extra polish.
Total		**175 h**	

> I have no academic commitments during May to August 2026. I can put in 15 to 20 hours per week. I will be available for weekly calls and will use `ml4-sci@cern.ch` for all questions.

4.1 Potential blockers and how I will handle them

I want to be honest about where things could go slower than planned, and what I will do about it.

- **Per-event fitting is slow.** Fitting one INR per event takes 1 to 2 minutes on a GPU. For 50k events that does not scale. My plan: start with a 5k subset for all experiments, then use the hyper-network (Phase 2) for full-dataset inference. If the hyper-network proves harder to

train than expected, I will report per-event results on the subset and document the scaling limitation clearly.

- **Hyper-network instability.** Predicting INR weights directly is known to be tricky (the loss landscape is complex). If I cannot get stable training in 2 weeks, I will switch to predicting a lower-dimensional latent code and using it for classification only, rather than full weight prediction. This still answers the core research question.
- **Colab GPU limits.** Free Colab has session limits. I will structure all training to save checkpoints every epoch so nothing is lost on disconnects.
- **Dataset or dependency issues.** I will flag anything unexpected on `ml4-sci@cern.ch` within 24 hours rather than spending days debugging alone.

5. Deliverables

- INR model (Fourier Feature + MLP) integrated into the GENIE codebase.
- Hyper-network for fast inference on new events.
- Reconstruction results: MSE, PSNR, SSIM per channel.
- Classification results: AUC and ROC curves vs. AE and GNN baselines.
- Ablation study over encoding type, Fourier scale, and depth.
- Upsampling demo: same INR weights queried at 125x125 and 250x250.
- Jupyter notebooks reproducing all figures with one command.
- Docstrings on every function, type hints throughout, and a README covering installation, usage, and how to reproduce results.
- Unit tests for the core components: coordinate encoding, INR forward pass, and data preprocessing.
- Final written report with results and next steps.

6. About Me

6.1 What I found in the GENIE codebase

Before writing this proposal I looked through the ML4SCI GENIE repository and the existing project code. A few things I noticed:

- The evaluation tasks use the `energyflow` library to load the quark/gluon dataset. I set this up locally and confirmed the data loading works before writing any proposal code.
- The existing ML4SCI notebooks use standard PyTorch patterns. My INR implementation will follow the same structure so it fits naturally into the codebase rather than introducing a different style.
- There is no existing INR baseline in the GENIE repo. This means I will need to write the full pipeline from scratch, which I have accounted for in Phase 1 of the timeline.

One open question I have for mentors: is there a preferred way to handle the per-event fitting loop at scale (e.g. parallelising across events, or storing fitted weights on disk between runs)? I have a plan, but I would like to align with whatever approach fits best with the existing infrastructure.

6.2 Evaluation tasks I already completed

> All three ML4SCI evaluation tasks are done and working. Code is at github.com/V-SOUMYA and linked in the Google Form submission.

Common Task 1: Autoencoder. Trained a convolutional autoencoder (3.7M parameters) on 5000 three-channel jet images. The encoder compresses each event to a 128-dimensional vector. The decoder rebuilds it back to 125x125.

Channel	MSE	PSNR (dB)	SSIM
Tracks	0.00000	69.06	0.9998
ECAL	0.00000	58.59	0.9986
HCAL	0.00001	51.07	0.9931
Overall	0.00000	55.08	–

PSNR above 40 dB is considered excellent. All three channels are well above that threshold.

Common Task 2: GNN classifier. Converted jet images to point clouds (non-zero pixels only). Built kNN graphs and trained an EdgeConv GNN for quark/gluon classification. Used early stopping to prevent overfitting.

Metric	Score
Test Accuracy	0.6800
Test ROC-AUC	0.7316

Specific Task 2: INR experiments. Fit a FourierINR (Fourier Feature Network + 4-layer MLP) to individual jet events. Each event is represented as a continuous function. Demonstrated resolution independence by querying the same weights at 64x64, 125x125, and 250x250.

Channel	MSE	PSNR (dB)	SSIM
Tracks	0.00000	55.50	0.9716
ECAL	0.00000	55.06	0.9773
HCAL	0.00001	53.45	0.9713
Overall	0.00000	54.67	0.9734

These are not just checkboxes. The autoencoder and GNN are my two baselines. The INR achieves comparable SSIM (0.97) to the autoencoder despite being a continuous function rather than a grid, and without any encoder training on the full dataset. I start the summer with all three working and directly comparable.

6.3 Relevant projects

Fruit Detection (B.Tech, 2022). Built a MobileNetV2 classifier to detect rotten fruit. Handled class imbalance, added dropout, and evaluated with precision, recall, and F1. Reached 99.6% validation accuracy. Won Most Innovative Project at the JUET Builds hackathon. This is where I learned to properly debug a training loop.

GenAI tools at Accenture (2024). Built Python automation tools for Amazon Q and presented findings to Amazon's engineering team directly. I learned to write code that other people can actually read and maintain. That habit matters for open-source work.

Coordinate MLP (personal). Read the NeRF and Fourier Feature papers, then built a small coordinate network myself to fit 2D scalar fields. This is the most directly relevant project I have done before this proposal.

6.4 Why I will finish this

The plan is deliberately conservative. Phases 1 and 2 alone produce a complete, submittable result. Phases 3 and 4 are additions. If something takes longer than expected, I have buffer time. If things go faster, I go deeper on ablations.

I am not trying to do everything. I am trying to do the important thing well.

6.5 Long-term connection

This project fits directly into what I want to do after my M.S. I am interested in scientific ML: using deep learning not just on benchmark datasets, but on real measurement problems where the representation you choose actually changes what physics you can discover.

INRs are still an open research area. Nobody has definitively shown whether they beat conventional discrete representations for HEP data. This project produces a real answer to that question on a public dataset. That is something I can build on, publish from, or carry into a research position.

Beyond the result itself, contributing to an open-source scientific ML project teaches me things that coursework does not: how to write code that other researchers can use, how to communicate technical decisions to domain experts who are not ML people, and how to scope a project so it actually ships. Those are exactly the skills I want to develop this summer.

7. References

[1] Tancik, M. et al. (2020). *Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains*. NeurIPS 2020.

[2] Sitzmann, V. et al. (2020). *Implicit Neural Representations with Periodic Activation Functions*. NeurIPS 2020.

[3] Mildenhall, B. et al. (2020). *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. ECCV 2020.

[4] Dupont, E. et al. (2022). *From data to functa*. ICML 2022.

[5] ML4SCI GENIE Project. https://ml4sci.org